

YOLO Dataset Configuration

Introduction

During this week's task, I explored how YOLOv26 datasets are structured and how meta configuration files help in training object detection models. I analyzed the dataset folders, YAML configuration file, and label files to understand how the model reads and interprets data.

Dataset Structure

A YOLOv26 dataset is organized into separate folders for images and labels. Inside both, there are subfolders for training, validation, and testing.

The structure looks like this:

```
dataset/
├── images/
│   ├── train/
│   ├── val/
│   └── test/
├── labels/
│   ├── train/
│   ├── val/
│   └── test/
├── data.yaml
├── train.txt
└── val.txt
```

From my understanding, this structure is important because the model needs both the image and its corresponding label file to learn properly. Each image in the images folder has a matching label file in the labels folder.

YAML Configuration File (data.yaml)

The YAML file is the main meta configuration file in YOLOv26. It tells the model where the dataset is located and what classes are present.

Example:

path: /dataset
train: images/train
val: images/val

nc: 3
names: ['drone', 'bird', 'plane']

From my analysis:

- “path” defines the root dataset folder
- “train” and “val” tell where training and validation images are stored
- “nc” is the number of classes
- “names” contains class labels

This file acts like a guide for the model to understand the dataset.

Label Files

Each image has a corresponding label file in .txt format. These files contain object information.

Format:

<class_id> <x_center> <y_center>

Example:

0 0.45 0.52 0.30 0.20

After studying this, I understood:

- class_id represents the object category (from YAML file)
- x_center and y_center represent the center of the object
- width and height define the size of the bounding box
- All values are normalized between 0 and 1

This format helps the model locate objects precisely inside the image.

Train.txt and Val.txt Files

These files contain paths of images used for training and validation.

Example:

images/train/img1.jpg

images/train/img2.jpg

These files help the model quickly access the dataset without scanning folders manually.

My Understanding of Meta Files

From this task, I understood that meta files are essential for:

- Defining dataset structure
- Linking images with labels
- Providing class information
- Helping the model load data correctly

If these files are incorrect, the model may not train properly or may produce wrong predictions.

Tools Used for Labeling

To create labeled datasets, tools like:

- Label Studio
- labellmg

can be used. These tools help in drawing bounding boxes and generating label files in YOLO format.

Conclusion

Through this exploration, I learned how YOLOv26 uses configuration files and dataset structure to perform object detection. Proper dataset preparation and correct meta files are very important for achieving good model accuracy. This task helped me understand the practical side of training object detection models.

References

Ultralytics YOLO Documentation: <https://docs.ultralytics.com>

Label Studio Documentation: <https://labelstud.io/guide/>